

ITAS170 — Lab 1

The shell is the cockpit

Time: one class, Monday 9:00–2:00 · **Marks:** 20

Work from a terminal, and keep the session log. Every tool in this course is driven from a terminal, and every conversation you have with an AI assistant about it is recorded in a file called `LOG.md`. Both habits start today.

Work in `~/code/ITAS170/week01/`. Everyone in the class uses the same folder, which means every command on the projector is the command you should be typing.

Get the starter files first: download `itas170-week01-student.zip` from the week 1 section on Moodle and unzip it into `~/code/ITAS170/week01/`. Before you start, `ls` should show `access.log`, `toolbox.sh` and `LOG-template.md`. If it does not, put your hand up before anything else.

How the day runs

Time		
9:00–9:40	Intro A	Where these tools came from (15 min); then programs talk in text: pipes, redirection, the manual
9:40–10:30	Session A — Parts 1-2	~50 min hands-on
10:30–11:00	Break	
11:00–11:40	Intro B	Permissions, PATH, a script — and an assistant you check
11:40–12:30	Session B — Parts 3-5	Instructor breaks for lunch around noon
12:30–2:00	Finish + demos	Help, then show+tell marking as you finish

You are marked in person, and demos run all day — call the instructor over as soon as you finish a part, do not save it for 2:00. **6 of the 20 marks are for explaining your work:** what a command does, why that option, what would change if you swapped two stages of a pipe. Working commands you cannot explain do not earn those marks.

Part 1 — Toolbox check · 20 min

Open a terminal with `Ctrl` + `Alt` + `T`, go to your week folder, and run the check script:

```
cd ~/code/ITAS170/week01
bash toolbox.sh
```

It prints one line per tool this term uses, then a few version lines. Paste the **whole output** into `toolbox.txt`. A `MISSING` line is not your problem to fix today — it is the instructor's — but it must be in the file.

Then open `toolbox.sh` in VS Code and **read it**. It is a shell script: a file of commands the shell runs top to bottom. You will write a much shorter one in Part 3.

Your seat. Every student in this course has a seat on the department's Claude Team plan this term. Open the Claude desktop app, sign in with the account the instructor gave you, and click the **Code** tab: this is the agent you will drive from week 6, and the chat you may use from today. In the terminal, `claude --version` should now print a version instead of a `MISSING` line — if it printed `MISSING` in the output above, run the script again after signing in and paste the new output. Then accept the invitation to the class **Project** ("ITAS 170"): its instructions are the course's AI policy, and everything you put in a shared Project is on a shared desk — treat it that way.

Hand in: `toolbox.txt` — the pasted output — and the Code tab open on your screen at the demo.

Part 2 — The pipe hunt · 35 min · hands-only

`access.log` is four days of a small web server's traffic, 2400 lines, one request per line. Here is one line, with the pieces you will need:

```
142.232.10.25 - - [10/Sep/2026:00:41:44 -0700] "GET / HTTP/1.1" 200 5316 "-" "Mozilla/
5.0 ..."
    ^ client           ^ date : hour           ^ method ^ path           ^ status ^
bytes
```

Split on spaces, the **client** is field 1, the **path** is field 7, the **status code** is field 9 and the **bytes** are field 10. Split on colons, the **hour** is field 2. That is everything you need to know about the format.

Answer these ten questions **using only pipes of the commands from this morning** — `cat`, `head`, `tail`, `wc`, `grep`, `cut`, `sort`, `uniq`. For each, write the command you used and the answer it printed in `hunt.md`:

1. How many requests are in the log?

2. How many requests were answered with status `404`?
3. Which five clients made the most requests, and how many each?
4. Which hour of the day is busiest across the whole log?
5. How many **distinct** paths were requested?
6. Which path produced the most 404s, and how many? — This site is not WordPress. Look at who is asking, and add one sentence about what you think is going on.
7. What was the largest single response — its path and its size in bytes?
8. How many requests came from the Googlebot crawler? Then count lines containing **bot case-insensitively** and explain, in one sentence, why the two numbers differ.
9. How many requests happened on 12 September?
10. How many `POST` requests are there, and how many of those were to `/wp-login.php`?

Two rules make this part worth doing. **Type the commands yourself** — this is a hands-only part; an assistant may explain a command to you but may not write your pipeline. And **build each pipe one stage at a time**: run the first command, look at the output, add `| head`, look again, add the next stage. If a number looks wrong, it probably is — question 8 is designed to look right and be wrong.

Hand in: `hunt.md` — ten commands, ten answers, and the two sentences (questions 6 and 8).

Part 3 — A script on your PATH · 30 min

Make a folder for your own commands and write your first one:

```
mkdir -p ~/code/ITAS170/bin
cd ~/code/ITAS170/bin
```

Create `hello-tools` (no extension) in VS Code, exactly this:

```
#!/usr/bin/env bash
# hello-tools — my first command.
echo "Hello from $USER on $(hostname)"
echo "You are standing in: $(pwd)"
echo "The time is $(date '+%H:%M') and your shell is $SHELL"
```

Now run it, and read what happens:

```
./hello-tools
```

Permission denied is the correct result. A file is not a program until you say so:

```
ls -l hello-tools
chmod +x hello-tools
ls -l hello-tools
./hello-tools
```

Read the two `ls -l` lines against each other — one letter changed. Then go home and try again:

```
cd ~
hello-tools
```

command not found is also correct. The shell only looks for commands in the folders listed in `PATH`. Look at the list:

```
echo "$PATH" | tr ':' '\n'
```

Your folder is not in it. Add **one line** to the end of `~/.bashrc`:

```
export PATH="$HOME/code/ITAS170/bin:$PATH"
```

Open a **new** terminal (or run `source ~/.bashrc`), then from your home folder:

```
hello-tools
which hello-tools
```

Hand in: `bin/hello-tools`, and `week01/path.md` containing: the `Permission denied` line, the two `ls -l` lines, the `command not found` line, the `which` output after the fix, and **one sentence** saying why the command was not found before the fix.

Part 4 — Explain, verify, run · 40 min

Now the assistant. Pick **five commands** from your hunt (a whole stage each — `sort -rn`, `uniq -c`, `cut -d' ' -f9`, `grep -c '^404$'`, `sort -k10,10n` are good choices). For each one:

1. **Ask** your assistant to explain it, option by option. Use the same prompt every time and write that prompt in `LOG.md`.
2. **Verify** against the manual: `man sort`, or `sort --help`, and read the entry for each option the assistant mentioned.

3. **Record** it in `explain.md` as a table row: the command · what the assistant said · what the manual says · your verdict (agree, differs, or incomplete).

Then one paragraph at the bottom: **the one discrepancy that mattered most**, and what would have gone wrong if you had believed the assistant. If all five genuinely agreed, say so — and name the one thing the manual told you that the assistant left out. There is always one.

This is the ritual for the whole term: **explain → verify → run**. An assistant is fluent, fast, and sometimes wrong in a way that sounds right. The manual is dull and correct. You need both.

Hand in: `explain.md` — five rows and the paragraph.

Part 5 — The first session log · 25 min

Copy the template and fill it in:

```
cd ~/code/ITAS170/week01
cp LOG-template.md LOG.md
```

`LOG.md` needs **at least two entries** from today: the Part 4 conversation, and one more — any other question you asked an assistant, about anything in this lab. Each entry has four lines: what you asked, what came back **in your words**, what you verified it against, and what you would ask differently.

Keep it honest. "It told me X, I did not check" is a valid entry and a better one than a paragraph of praise. Next week this file goes into git, and from then on your conversations with these tools have a history.

Hand in: `LOG.md` with two or more entries.

Marking — 20 marks

	Component	Marks
C1	Toolbox check — the whole output, pasted	2
C2	The pipe hunt — ten commands, ten answers, two sentences	5
C3	A script on your PATH — runs from <code>~</code> , <code>path.md</code> complete	3
C4	Explain, verify, run — five rows and the discrepancy paragraph	3

Component		Marks
C5	The session log — two honest entries	1
DU	Demonstrates understanding — marked in person	6
		20

DU is awarded from your show+tell demo, not from your files. Expect to be asked what one stage of a pipe does, and what the output would be if two stages swapped places — then to run it and see.

Submitting

Zip the whole week01 folder — toolbox.txt, hunt.md, path.md, explain.md, LOG.md — plus bin/hello-tools, and upload it to the week 1 section on Moodle by **Sunday 20 September**. The demo was the mark; the zip is the record.

On using AI for this lab

This course teaches you to work with AI assistants, so you are expected to use one today — in Part 4 it is the whole point. Two rules, and they hold all term: **everything you ask goes in LOG.md**, and **Part 2 is hands-only** — the assistant may explain a command you have already tried, but you type every pipeline yourself. You cannot check a tool doing something you have never done. If your assistant is down, the manual pages are the assistant.

One more thing, because the same class takes ITAS 185 on Wednesday: the rules there are different, on purpose. Learning to drive an assistant here is not permission to hand it your Python there.

If something goes wrong

What you see	What it means	What to do
bash: ./hello-tools: Permission denied	The file exists but is not marked executable. Expected once, in Part 3.	<code>chmod +x hello-tools</code> . If it happens to a file you did not write, ask before making it executable.
hello-tools: command not found from your home folder	Your <code>bin</code> folder is not on <code>PATH</code> , or you edited <code>.bashrc</code> but this terminal started before the edit.	Open a new terminal, or <code>source ~/.bashrc</code> . Check with <code>echo "\$PATH" tr ':' '\n'</code> — your folder should be the first line.
/usr/bin/env: 'bash\r': No such file or directory	The file has Windows line endings — it was edited on Windows, or pasted from a page that had them.	In VS Code, click CRLF in the bottom bar and change it to LF, save, run again.
No such file or directory for access.log	You are not in <code>~/code/ITAS170/week01</code> .	<code>pwd</code> , then <code>cd</code> there. <code>ls</code> should show the log.
Your <code>uniq -c</code> counts look too small, or the same value appears twice	<code>uniq</code> only collapses adjacent repeats, and the input was not sorted.	Put <code>sort</code> before <code>uniq -c</code> . Read <code>man uniq</code> — the first line says so.
<code>cut</code> prints the wrong field, or an empty line	The delimiter is wrong (<code>cut -d ' '</code> versus the default <code>tab</code>), or the field number is off by one.	Count the spaces in one line by hand. Fields start at 1, not 0.
Question 8 gives you 374 and you expected 335	<code>bot</code> matches inside other words.	<code>grep 'robots.txt' access.log wc -l</code> and think about where else the letters b-o-t appear. That is the answer to the question.
No manual entry for X	Some tools ship no <code>man</code> page.	<code>X --help</code> , or <code>tldr X</code> if it is installed.
The assistant explains a command confidently and the manual says something else	This is the lab, not a problem.	Write it down in <code>explain.md</code> . That row is worth more than five agreements.

If you are stuck for more than five minutes on anything above, call the instructor over — that is what the lab time is for.